

Observation Task 2

1. Inheritance

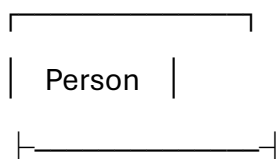
Steps

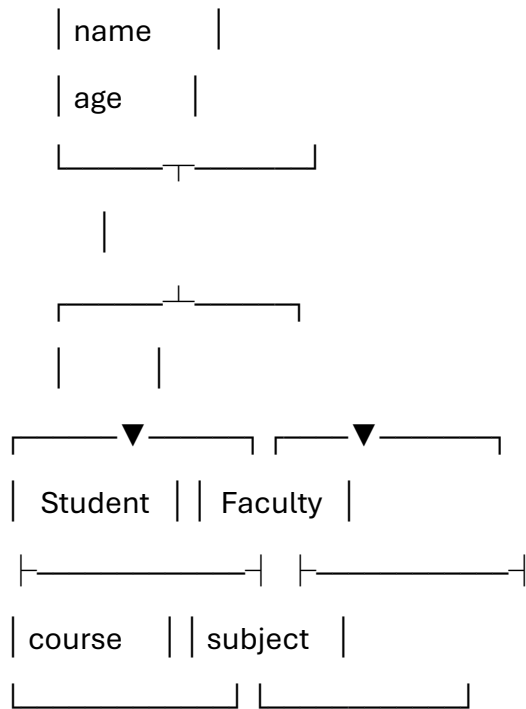
1. Create a Person class with name and age.
2. Create a Student class that extends Person.
3. Add course property to Student.
4. Create a Faculty class that extends Person.
5. Add subject property to Faculty.
6. Create objects of Student and Faculty.
7. Access the inherited properties using the objects.
8. Display the details.

Algorithm

1. Start.
2. Create the Person class.
3. Declare name and age.
4. Create Student extending Person.
5. Declare course in Student.
6. Create Faculty extending Person.
7. Declare subject in Faculty.
8. Create objects s and f.
9. Display student details.
10. Display faculty details.
11. Stop.

UML Diagram





Code

```
class Person {
```

```
    String name = "Naveen";
```

```
    int age = 19;
```

```
}
```

```
class Student extends Person {
```

```
    String course = "CSM";
```

```
}
```

```
class Faculty extends Person {
```

```
    String subject = "Java";
```

```
}
```

```
public class Inheritance {
```

```
    public static void main(String[] args) {
```

```
        Student s = new Student();
```

```
Faculty f = new Faculty();

System.out.println("Student Name: " + s.name);
System.out.println("Student Age: " + s.age);
System.out.println("Student Course: " + s.course);

System.out.println("Faculty Name: " + f.name);
System.out.println("Faculty Age: " + f.age);
System.out.println("Faculty Subject: " + f.subject);
}
}
```

Sample Output

Student Name: Naveen

Student Age: 19

Student Course: CSM

Faculty Name: Naveen

Faculty Age: 19

Faculty Subject: Java

2. Class and Object

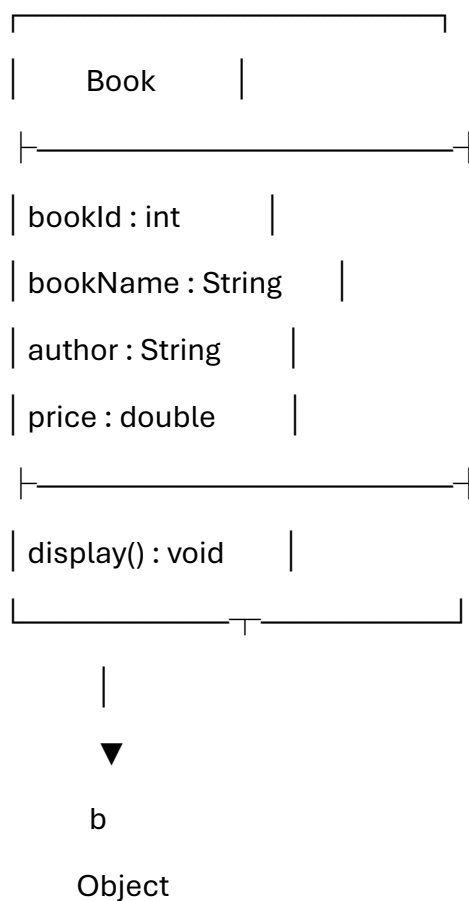
Steps

1. Create a class named Book.
2. Declare book ID, book name, author, and price.
3. Create a display() method.
4. Create an object of the Book class.
5. Assign values to the object.
6. Call the display() method.
7. Display the book details.

Algorithm

1. Start.
2. Create the Book class.
3. Declare book properties.
4. Create the display() method.
5. Create object b.
6. Assign values to the object.
7. Call display().
8. Display book details.
9. Stop.

UML Diagram



Code

```
class Book {  
    int bookId;
```

```
String bookName;

String author;

double price;


void display() {

    System.out.println("Book ID: " + bookId);

    System.out.println("Book Name: " + bookName);

    System.out.println("Author: " + author);

    System.out.println("Price: " + price);

}

}


public class ClassObject {

    public static void main(String[] args) {

        Book b = new Book();

        b.bookId = 101;

        b.bookName = "Java Programming";

        b.author = "James Gosling";

        b.price = 450.50;

        b.display();

    }

}
```

Sample Output

Book ID: 101

Book Name: Java Programming

Author: James Gosling

Price: 450.5

3. Constructor and Encapsulation

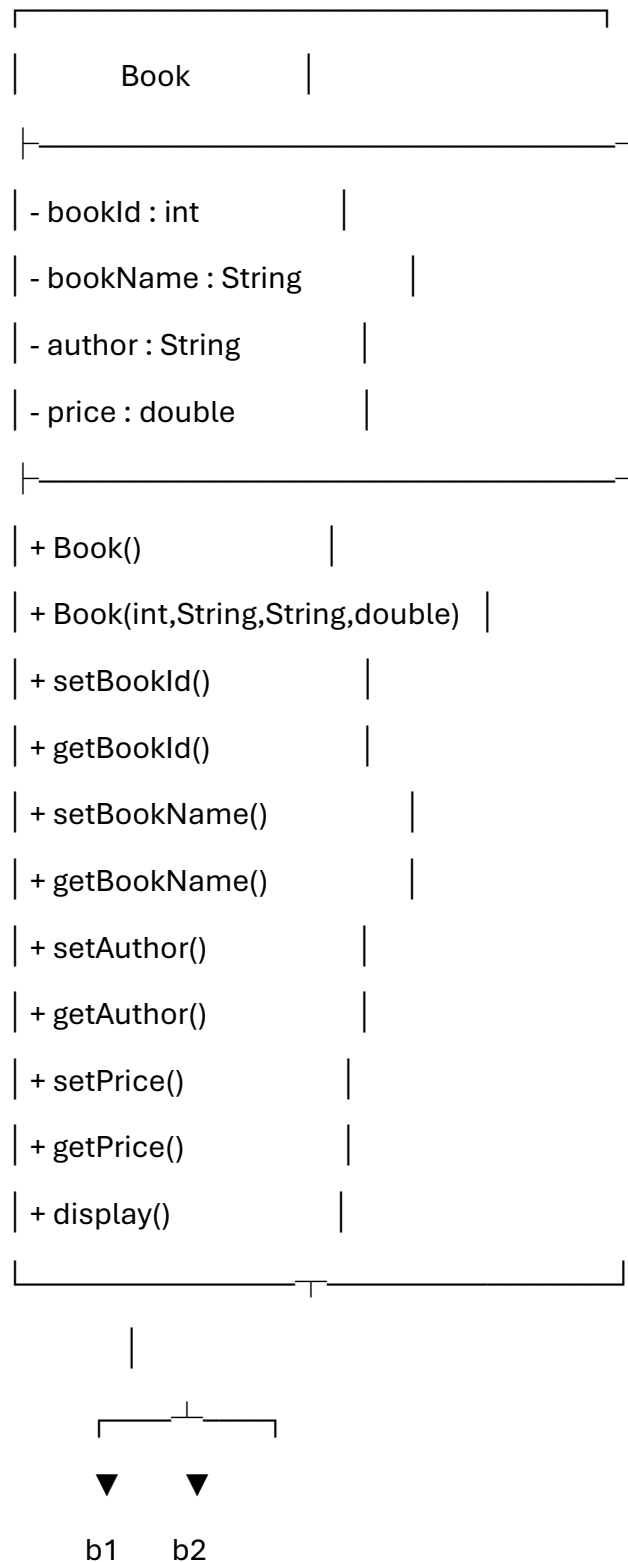
Steps

1. Create a Book class.
2. Declare private variables.
3. Create a default constructor.
4. Create a parameterized constructor.
5. Create setter and getter methods.
6. Create a display() method.
7. Create two objects.
8. Initialize the first object using setters.
9. Initialize the second object using the parameterized constructor.
10. Display both objects.

Algorithm

1. Start.
2. Create the Book class.
3. Declare private book properties.
4. Create a default constructor.
5. Create a parameterized constructor.
6. Create getter and setter methods.
7. Create objects b1 and b2.
8. Set values for b1.
9. Pass values through constructor for b2.
10. Display both books.
11. Stop.

UML Diagram



Code

```

class Book {

    private int bookId;
  
```

```
private String bookName;
```

```
private String author;
```

```
private double price;
```

```
Book() {
```

```
    bookId = 0;
```

```
    bookName = "";
```

```
    author = "";
```

```
    price = 0.0;
```

```
}
```

```
Book(int bookId, String bookName, String author, double price) {
```

```
    this.bookId = bookId;
```

```
    this.bookName = bookName;
```

```
    this.author = author;
```

```
    this.price = price;
```

```
}
```

```
void setBookId(int bookId) {
```

```
    this.bookId = bookId;
```

```
}
```

```
int getBookId() {
```

```
    return bookId;
```

```
}
```

```
void setBookName(String bookName) {
```

```
    this.bookName = bookName;
```

```
}
```

```
String getBookName() {  
    return bookName;  
}
```

```
void setAuthor(String author) {  
    this.author = author;  
}
```

```
String getAuthor() {  
    return author;  
}
```

```
void setPrice(double price) {  
    this.price = price;  
}
```

```
double getPrice() {  
    return price;  
}
```

```
void display() {  
    System.out.println("Book ID: " + getBookId());  
    System.out.println("Book Name: " + getBookName());  
    System.out.println("Author: " + getAuthor());  
    System.out.println("Price: " + getPrice());  
}
```

```
}
```

```
public class ConstructorEncapsulation {
```

```
    public static void main(String[] args) {
```

```
        Book b1 = new Book();
```

```
        b1.setBookId(101);
```

```
        b1.setBookName("Java Programming");
```

```
        b1.setAuthor("James Gosling");
```

```
        b1.setPrice(450.50);
```

```
        b1.display();
```

```
        System.out.println();
```

```
        Book b2 = new Book(102, "Python Programming", "Guido van Rossum", 500.00);
```

```
        b2.display();
```

```
    }
```

```
}
```

Sample Output

Book ID: 101

Book Name: Java Programming

Author: James Gosling

Price: 450.5

Book ID: 102

Book Name: Python Programming

Author: Guido van Rossum

Price: 500.0

4. Polymorphism

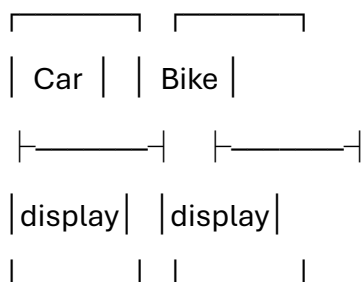
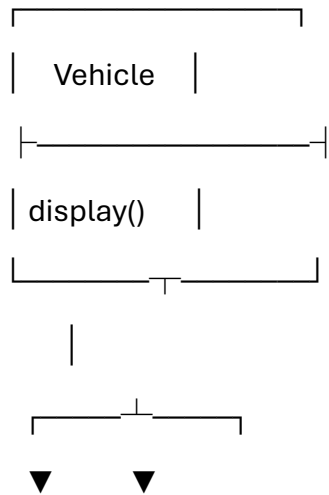
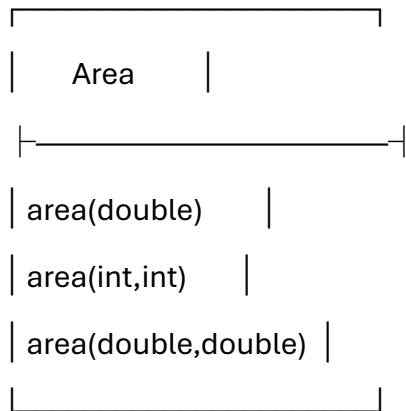
Steps

1. Create an Area class.
2. Create three area() methods with different parameters.
3. This demonstrates method overloading.
4. Create a Vehicle class with display().
5. Create Car and Bike classes extending Vehicle.
6. Override the display() method.
7. Create objects using parent class references.
8. Call the overridden methods.
9. Display the results.

Algorithm

1. Start.
2. Create Area class.
3. Create three overloaded area() methods.
4. Create Vehicle class.
5. Create Car and Bike classes extending Vehicle.
6. Override display() in both classes.
7. Create Area object.
8. Calculate circle, rectangle, and triangle areas.
9. Create Vehicle references for Car and Bike.
10. Call display().
11. Stop.

UML Diagram



Code

```
class Area {  
  
    double area(double radius) {  
        return Math.PI * radius * radius;  
    }  
}
```

```
}
```

```
int area(int length, int breadth) {
```

```
    return length * breadth;
```

```
}
```

```
double area(double base, double height) {
```

```
    return 0.5 * base * height;
```

```
}
```

```
}
```

```
class Vehicle {
```

```
    void display() {
```

```
        System.out.println("This is a vehicle");
```

```
    }
```

```
}
```

```
class Car extends Vehicle {
```

```
    void display() {
```

```
        System.out.println("This is a car");
```

```
    }
```

```
}
```

```
class Bike extends Vehicle {
```

```
    void display() {
```

```
        System.out.println("This is a bike");
    }
}

public class Polymorphism {

    public static void main(String[] args) {

        Area a = new Area();

        System.out.println("Area of Circle: " + a.area(5.0));
        System.out.println("Area of Rectangle: " + a.area(10, 5));
        System.out.println("Area of Triangle: " + a.area(10.0, 5.0));

        System.out.println();

        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();

        v1.display();
        v2.display();
    }
}
```

Sample Output

Area of Circle: 78.53981633974483

Area of Rectangle: 50

Area of Triangle: 25.0

This is a car

This is a bike

5. Abstract Class and Interface

Steps

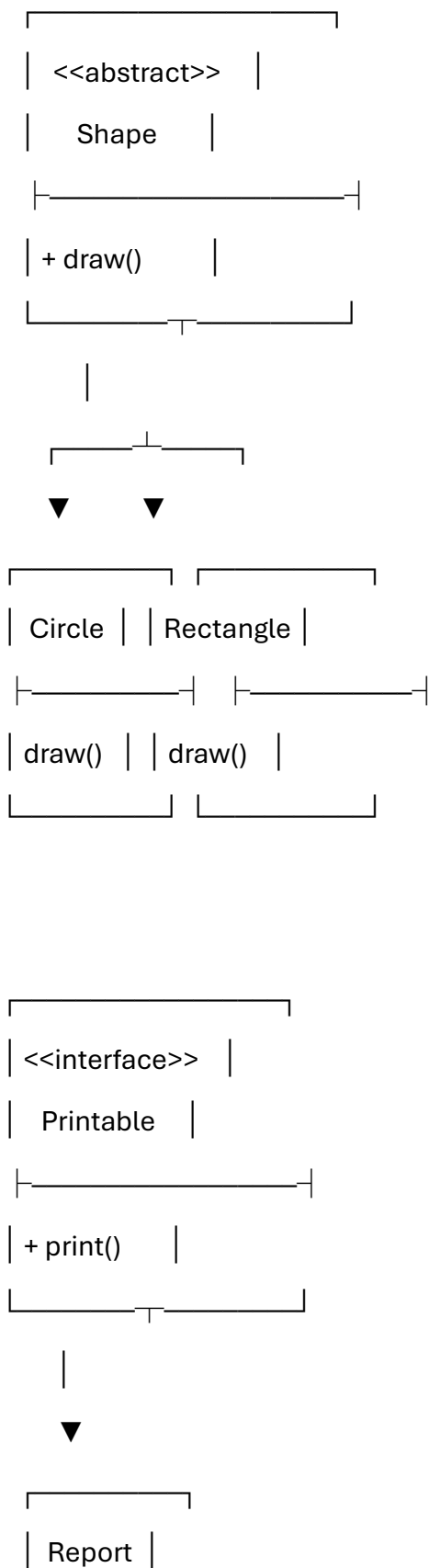
1. Create an abstract class Shape.
2. Declare an abstract method draw().
3. Create Circle extending Shape.
4. Implement draw() in Circle.
5. Create Rectangle extending Shape.
6. Implement draw() in Rectangle.
7. Create an interface Printable.
8. Declare print() in the interface.
9. Create Report implementing Printable.
10. Implement print() in Report.
11. Create objects and call the methods.

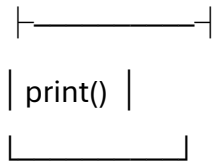
Algorithm

1. Start.
2. Create abstract class Shape.
3. Declare abstract method draw().
4. Create Circle and Rectangle.
5. Implement draw() in both classes.
6. Create Printable interface.
7. Declare print() method.
8. Create Report implementing Printable.
9. Implement print().
10. Create objects.
11. Call the methods.

12. Stop.

UML Diagram





Code

```
abstract class Shape {  
    abstract void draw();  
}
```

```
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

```
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Drawing Rectangle");  
    }  
}
```

```
interface Printable {  
    void print();  
}
```

```
class Report implements Printable {  
    public void print() {  
        System.out.println("Printing Report");  
    }  
}
```

```
}
```

```
public class AbstractInterface {  
    public static void main(String[] args) {  
        Shape c = new Circle();  
        Shape r = new Rectangle();  
  
        c.draw();  
        r.draw();  
  
        Report report = new Report();  
        report.print();  
    }  
}
```

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\observationprograms\Task2> java C  
nheritance  
Student Name: Naveen  
Student Age: 19  
Student Course: CSM  
Faculty Name: Naveen  
Faculty Age: 19  
Faculty Subject: Java
```

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\observationprograms\Task2> java C  
lassObject  
Book ID: 101  
Book Name: Java Programming  
Author: James Gosling  
Price: 450.5
```

```
PS C:\Users\navee\OneDrive\Desktop\javaprograms\observationprograms\Task2> java C  
onstructorEncapsulation  
Book ID: 101  
Book Name: Java Programming  
Author: James Gosling  
Price: 450.5  
  
Book ID: 102  
Book Name: Python Programming  
Author: Guido van Rossum  
Price: 500.0
```

```
PS C:\Users\havee\OneDrive\Desktop\javaprograms\observationprograms\Task2> java P
olymorphism
Area of Circle: 78.53981633974483
Area of Rectangle: 50
Area of Triangle: 25.0

This is a car
This is a bike
```

```
PS C:\Users\havee\OneDrive\Desktop\javaprograms\observationprograms\Task2> java A
bstractInterface
Drawing Circle
Drawing Rectangle
Printing Report
```